SOUGATA SAHA
IT-A2-037 OOP ASSIGNMENT 5

Assignment 5

44. Two integers are taken from keyboard. Then perform division operation. Write a try block to throw an exception when division by zero occurs and appropriate catch block to handle the exception thrown.

Soln)

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two integers: ";
    cin >> a >> b;

    try {
        if (b == 0)
            throw "Division by zero!";
        cout << "Result: " << a / b << endl;
    } catch (const char* msg) {
        cout << "Error: " << msg << endl;
    }

    return 0;
}
```

45. Write a C++ program to demonstrate the use of try, catch block with the argument as an integer and string using multiple catch blocks.

Soln)

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main() {
    try {
        throw 10;
    } catch (int x) {
        cout << "Caught an integer: " << x << endl;
    } catch (string& s) {
        cout << "Caught a string: " << s << endl;
    }

    return 0;
}
```

46. Create a class with member functions that throw exceptions. Within this class, make a nested class to use as an exception object. It takes a single const char* as its argument, this represents a description string. Create a member function that throws this exception. (State this in the function s exception specification.) Write a try block that calls this function and a catch clause that handles the exception by displaying its description string.

Soln)

```
#include <iostream>
#include <string>
using namespace std;

class MyClass {
public:
    class Exception {
    private:
        const char* message;
    public:
        Exception(const char* msg) : message(msg) {}
        const char* getMessage() { return message; }
```

```
    };

    void throwException() throw(Exception) {
        throw Exception("Nested class exception!");
    }
};

int main() {
    MyClass obj;
    try {
        obj.throwException();
    } catch (MyClass::Exception& e) {
        cout << "Caught exception: " << e.getMessage() << endl;
    }

    return 0;
}
```

47. Design a class Stack with necessary exception handling.

Soln)

```
#include <iostream>
#include <vector>
using namespace std;

class Stack {
private:
    vector<int> data;
public:
    void push(int value) {
        data.push_back(value);
```

```
    }
    void pop() {
        if (data.empty())
            throw "Stack Underflow!";
        data.pop_back();
    }
    int top() {
        if (data.empty())
            throw "Stack is empty!";
        return data.back();
    }
};

int main() {
    Stack s;
    try {
        s.push(10);
        s.push(20);
        cout << "Top: " << s.top() << endl;
        s.pop();
        s.pop();
        s.pop();
    } catch (const char* msg) {
        cout << "Error: " << msg << endl;
    }

    return 0;
}
```

48. Write a Garage class that has a Car that is having troubles with its Motor. Use a function- level try block in the Garage class constructor to catch an exception (thrown

from the Motor class) when its Car object is initialized. Throw a different exception from the body of the Garage constructor s handler and catch it in main().

Soln)

```
#include <iostream>
using namespace std;

class Motor {
public:
   Motor() {
      throw "Motor error!";
   }
};

class Car {
public:
   Car() {
      Motor m;
   }
};

class Garage {
public:
   Garage() try : car() {
   } catch (...) {
      throw "Garage error!";
   }
private:
   Car car;
};

int main() {
```

```
    try {
        Garage g;
    } catch (const char* msg) {
        cout << "Error: " << msg << endl;
    }

    return 0;
}
```

49. Vehicles may be either stopped of running in a lane. If two vehicles are running in opposite direction in a single lane there is a chance of collision. Write a C++ program using exception handling to avoid collisions. You are free to make necessary assumptions.

Soln)

```
#include <iostream>
using namespace std;

void checkCollision(bool vehicle1Running, bool vehicle2Running) {
    if (vehicle1Running && vehicle2Running)
        throw "Collision detected!";
}

int main() {
    bool vehicle1Running = true;
    bool vehicle2Running = true;

    try {
        checkCollision(vehicle1Running, vehicle2Running);
    } catch (const char* msg) {
        cout << "Error: " << msg << endl;
    }
```

```
    return 0;
}
```

50. Write a template function max() that is capable of finding maximum of two things (that can be compared). Used this function to find (1) maximum of two integers, (ii) maximum of two complex numbers (previous code may be reused). Now write a specialized template function for strings (i.e. char *). Also find the maximum of two strings using this template function.

Soln)

```
#include <iostream>
#include <string>
using namespace std;

template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}

template <>
const char* max(const char* a, const char* b) {
    return (strcmp(a, b) > 0) ? a : b;
}

int main() {
    cout << "Max of 10 and 20: " << max(10, 20) << endl;

    const char* str1 = "Apple";
    const char* str2 = "Orange";
    cout << "Max of two strings: " << max(str1, str2) << endl;
```

```
    return 0;
}
```

51. Write a template function swap() that is capable of interchanging the values of two variables. Used this function to swap (1) two integers, (ii) two complex numbers (previous code may be reused). Now write a specialized template function for the class Stack (previous code may be reused). Also swap two stacks using this template function.

Soln)

```
#include <iostream>
#include <string>
using namespace std;

template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int a = 10, b = 20;
    swap(a, b);
    cout << "Swapped values: " << a << ", " << b << endl;

    string s1 = "Hello", s2 = "World";
    swap(s1, s2);
    cout << "Swapped strings: " << s1 << ", " << s2 << endl;

    return 0;
}
```

52. Create a C++ template class for implementation of Stack data structure. Create a Stack of integers and a Stack of complex numbers created earlier (code may be reused). Perform some push and pop operations on these stacks. Finally print the elements remained in those stacks.

Soln)

```
#include <iostream>
#include <vector>
using namespace std;

template <typename T>
class Stack {
private:
    vector<T> data;
public:
    void push(T value) {
        data.push_back(value);
    }
    void pop() {
        if (data.empty())
            throw "Stack Underflow!";
        data.pop_back();
    }
    void display() {
        for (const auto& elem : data)
            cout << elem << " ";
        cout << endl;
    }
};

int main() {
```

```
    Stack<int> intStack;
    intStack.push(1);
    intStack.push(2);
    intStack.display();

    Stack<string> stringStack;
    stringStack.push("Apple");
    stringStack.push("Orange");
    stringStack.display();

    return 0;
}
```